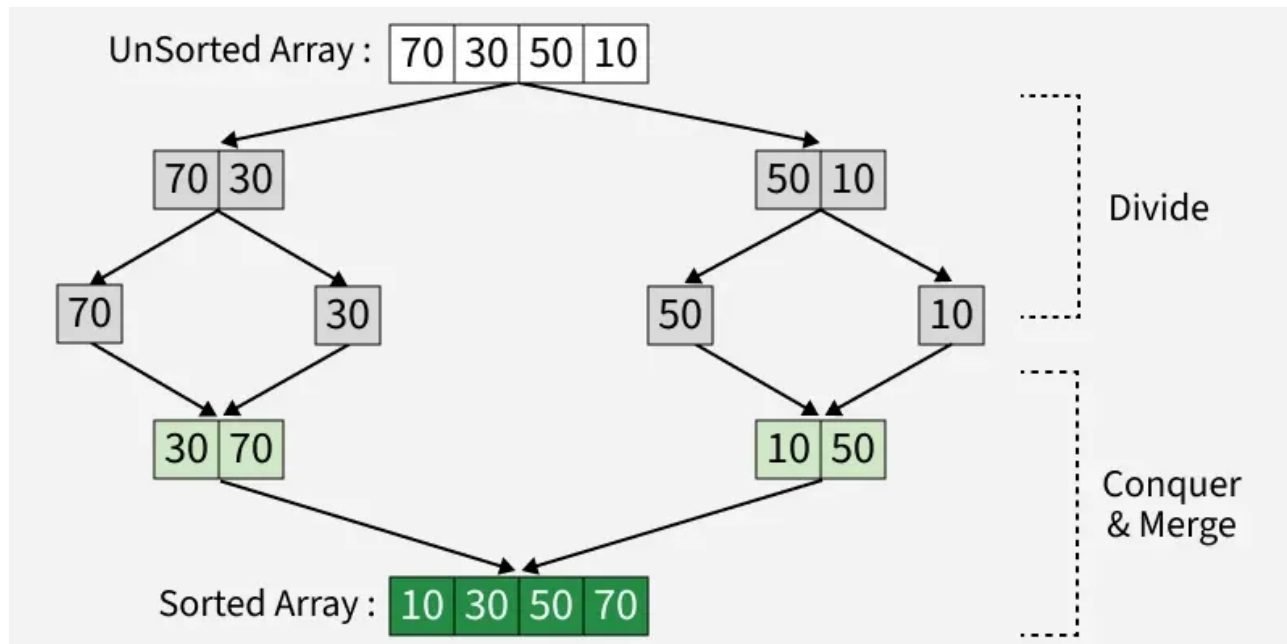


# Merge Sort

**Merge Sort** — bu samarali va barqaror saralash algoritmi bo‘lib, **Divide and Conquer** (Bo‘lib Olish va Yengish) tamoyiliga asoslanadi. Bu algoritm massivni rekursiv tarzda ikki bo‘lakka bo‘lib, har bir bo‘lakni alohida saralaydi va oxirida ularni birlashtirib, tartiblangan massivni hosil qiladi.



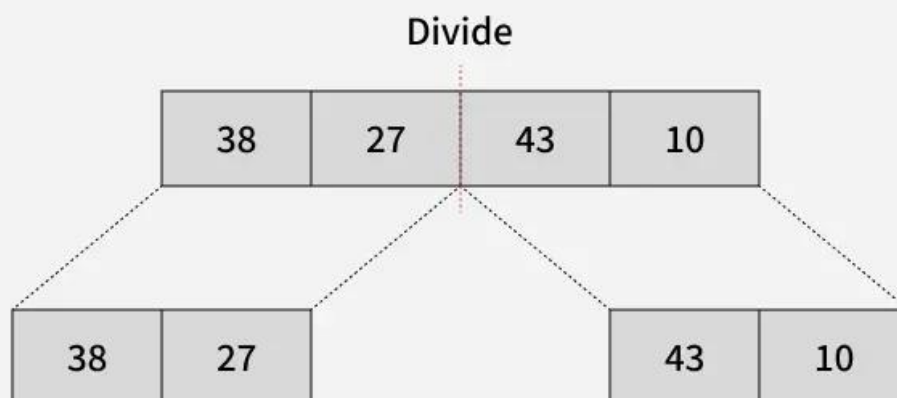
**Merge Sort** algoritmining ishlash prinsipi:

1. **Bo‘lish (Divide):** Massivni yoki ro‘yxatni rekursiv tarzda ikki bo‘lakka bo‘lib, har bir bo‘lakda faqat bitta element qolguncha bo‘lish davom ettiriladi;
2. **Yengish (Conquer):** Har bir kichik bo‘lak alohida **merge sort** algoritmi yordamida tartiblanadi;
3. **Birlashtirish (Merge):** Tartiblangan kichik bo‘laklar qaytadan tartiblangan holda birlashtiriladi. Bu jarayon davom etadi, toki har ikkala bo‘lakdagi barcha elementlar birlashtirilguncha.

**Merge Sort** algoritmi yordamida [38, 27, 43, 10] massivini tartiblash.

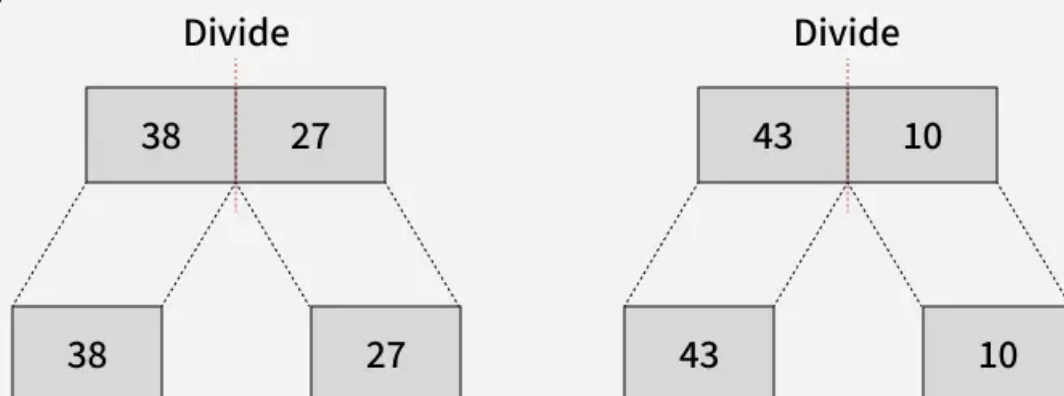
**01**  
Step

Splitting the Array into two equal halves



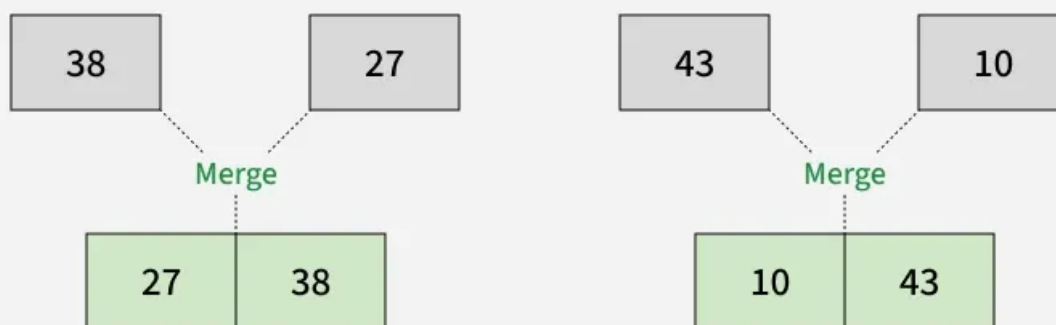
**02**  
Step

Splitting the subarrays into two halves



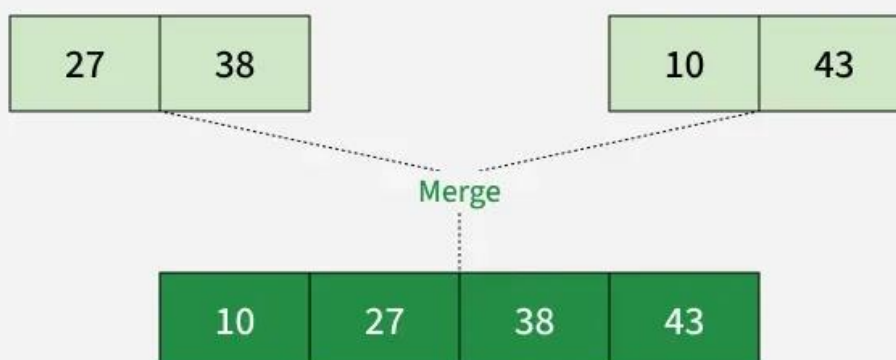
**03**  
Step

## Merging unit length cells into sorted subarrays



**04**  
Step

## Merging sorted subarrays into the sorted array



C++ kodi:

```
#include <iostream>
#include <vector>
using namespace std;

// arr[] ning ikki bo'lagini birlashtiradi.
// Birinchi bo'lak arr[left..mid]
// Ikkinchi bo'lak arr[mid+1..right]
void merge(vector<int>& arr, int left,
           int mid, int right){

    int n1 = mid - left + 1; // Chap bo'lakning uzunligi
    int n2 = right - mid;    // O'ng bo'lakning uzunligi

    // Yordamchi vektorlar yaratamiz
```

```

vector<int> L(n1), R(n2);

// L[] va R[] ga ma'lumotlarni ko'chiramiz
for (int i = 0; i < n1; i++)
    L[i] = arr[left + i]; // Chap bo'lakni L ga ko'chiramiz
for (int j = 0; j < n2; j++)
    R[j] = arr[mid + 1 + j]; // O'ng bo'lakni R ga ko'chiramiz

int i = 0, j = 0;
int k = left;

// Yordamchi vektorlarni qayta birlashtiramiz
// arr[left..right] ga
while (i < n1 && j < n2) {
    if (L[i] <= R[j]) {
        arr[k] = L[i];
        i++;
    }
    else {
        arr[k] = R[j];
        j++;
    }
    k++;
}

// Agar L[] da qolgan elementlar bo'lsa, ularni ko'chiramiz
while (i < n1) {
    arr[k] = L[i];
    i++;
    k++;
}

// Agar R[] da qolgan elementlar bo'lsa, ularni ko'chiramiz
while (j < n2) {
    arr[k] = R[j];
    j++;
    k++;
}
}

// begin – chap indeks va end – o'ng indeks
// arr ning tartiblanishi kerak bo'lgan bo'lagini ko'rsatadi
void mergeSort(vector<int>& arr, int left, int right){

    if (left >= right)
        return;

    int mid = left + (right - left) / 2; // O'rtadagi indeks
    mergeSort(arr, left, mid); // Chap bo'lakni tartiblaymiz
    mergeSort(arr, mid + 1, right); // O'ng bo'lakni tartiblaymiz
    merge(arr, left, mid, right); // Ikki bo'lakni birlashtiramiz
}

```

```

}

// Dastur kodi
int main(){

    vector<int> arr = {38, 27, 43, 10}; // Tartiblanadigan massiv
    int n = arr.size(); // Massivning uzunligi

    mergeSort(arr, 0, n - 1); // Massivni mergeSort yordamida tartiblaymiz
    for (int i = 0; i < arr.size(); i++)
        cout << arr[i] << " "; // Tartiblangan massivni chiqarish
    cout << endl;

    return 0;
}

```

Python kodi:

```

def merge(arr, left, mid, right):
    n1 = mid - left + 1 # Chap bo'lakning uzunligi
    n2 = right - mid # O'ng bo'lakning uzunligi

    # Yordamchi massivlarni yaratamiz
    L = [0] * n1
    R = [0] * n2

    # L[] va R[] ga ma'lumotlarni ko'chiramiz
    for i in range(n1):
        L[i] = arr[left + i] # Chap bo'lakni L ga ko'chiramiz
    for j in range(n2):
        R[j] = arr[mid + 1 + j] # O'ng bo'lakni R ga ko'chiramiz

    i = 0
    j = 0
    k = left

    # Yordamchi massivlarni qayta birlashtiramiz
    # arr[left..right] ga
    while i < n1 and j < n2:
        if L[i] <= R[j]:
            arr[k] = L[i]
            i += 1
        else:
            arr[k] = R[j]
            j += 1
        k += 1

    # Agar L[] da qolgan elementlar bo'lsa, ularni ko'chiramiz
    while i < n1:

```

```

    arr[k] = L[i]
    i += 1
    k += 1

    # Agar R[] da qolgan elementlar bo'lsa, ularni ko'chiramiz
    while j < n2:
        arr[k] = R[j]
        j += 1
        k += 1

def mergeSort(arr, left, right):
    if left < right:
        mid = (left + right) // 2 # O'rtadagi indeks

        mergeSort(arr, left, mid) # Chap bo'lakni tartiblaymiz
        mergeSort(arr, mid + 1, right) # O'ng bo'lakni tartiblaymiz
        merge(arr, left, mid, right) # Ikki bo'lakni birlashtiramiz

# Dastur kodi
if __name__ == "__main__":
    arr = [38, 27, 43, 10] # Tartiblanadigan massiv

    mergeSort(arr, 0, len(arr) - 1) # Massivni mergeSort yordamida tartiblaymiz
    for i in arr:
        print(i, end=" ") # Tartiblangan massivni chiqaramiz
    print()

```

## Merge Sortning Murakkablik Tahlili

### Vaqt Murakkabligi:

1. **Yaxshi holat (Best Case):**  $O(n \log n)$ 
  - Agar massiv allaqachon tartiblangan bo'lsa
2. **O'rtacha holat (Average Case):**  $O(n \log n)$ 
  - Agar massiv tasodifiy tartiblangan bo'lsa
3. **Eng yomon holat (Worst Case):**  $O(n \log n)$ 
  - Agar massiv teskari tartibda bo'lsa

### Qo'shimcha Xotira (Auxiliary Space): $O(n)$

Merge sort qo'shimcha xotira talab qiladi, chunki u har bir bo'lakni tartiblayotganda vaqtincha massivlarni yaratadi. Shu sababli, qo'shimcha xotira murakkabligi  $O(n)$  bo'ladi. Bu xotira yordamchi massivni saqlash uchun kerak bo'ladi.

### Merge Sortning qo'llanilishi:

- **Katta ma'lumotlarni tartiblashtirish (Sorting large datasets):** Merge Sort katta hajmdagi ma'lumotlarni samarali tartiblashtirish uchun ishlatiladi, chunki uning vaqt murakkabligi  $O(n \log n)$  bo'lib, juda katta ma'lumotlarni ham tezda tartiblaydi.

- **Tashqi tartiblashtirish (External sorting):** Agar ma'lumotlar hajmi xotiraga sig'masa, ya'ni fayl tizimida bo'lsa, merge sort tashqi tartiblashtirish algoritmi sifatida ishlatiladi. Bu holatda, ma'lumotlar diskda saqlanadi va merge sort yordamida bo'laklarga ajratilib, ularni birlashtiradi.
- Merge Sort, inversiyalarni hisoblash (Inversion counting), o'ngdagi kichik elementlarni hisoblash (Count Smaller on Right) va Surpasser Count kabi masalalarni samarali hal qilish uchun ishlatiladi.
- **Dasturlash tillaridagi kutubxonalarda foydalanish:** Merge Sort va uning varianti **TimSort** Python, Java Android va Swift dasturlash tillarida ishlatiladi. TimSort ko'proq barqarorlikni ta'minlaydi, bu esa **QuickSort**da mavjud emas. Java'da `Arrays.sort` QuickSort'dan foydalanadi, lekin **`Collections.sort`** MergeSortni ishlatadi.
- **Linked listlarni tartiblash:** Merge Sort, **Linked lists**ni tartiblash uchun afzal ko'rilgan algoritmdir, chunki linked listlarda merge jarayoni oson va samarali bajariladi.
- **Parallel ishlash:** Merge Sortni parallel ravishda bajarish mumkin, chunki biz har bir sub-massivni mustaqil ravishda tartiblab, keyin ularni birlashtirgan holda tartiblangan massivni hosil qilamiz.
- **Ikkita tartiblangan massivning birlashishi va kesishishi:** Merge Sortning **merge** funksiyasi, ikkita tartiblangan massivni samarali ravishda birlashtirish yoki kesish (union and intersection) kabi muammolarni hal qilishda ishlatiladi.

|    |           |                                                                                                                                                                                                                                                                       |
|----|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. | MergeSort | acmp.ru ЗАДАЧА №12 Армия<br><a href="https://acmp.ru/asp/do/index.asp?main=task&amp;id_course=2&amp;id_section=12&amp;id_topic=9&amp;id_problem=48">https://acmp.ru/asp/do/index.asp?main=task&amp;id_course=2&amp;id_section=12&amp;id_topic=9&amp;id_problem=48</a> |
| 2. |           | robocontest.uz Masala #0041 Massiv<br><a href="https://robocontest.uz/tasks/0041">https://robocontest.uz/tasks/0041</a>                                                                                                                                               |
| 3. |           | robocontest.uz Masala # 0075 Inversiyalar soni<br><a href="https://robocontest.uz/tasks/0075">https://robocontest.uz/tasks/0075</a>                                                                                                                                   |

### Армия

(Время: 0,5 сек. Память: 16 Мб Сложность: 58%)

Всем известно, что в армии без строевой подготовки и порядка дело не обходится и за этим там строго следят. Однажды утром сержант построил всех своих подчиненных в  $K$  рядов по  $N$  человек в каждом, но оказалось, что солдаты выстроились не по росту, и поэтому сержант решил их наказать. Солдаты должны были выстроиться по росту в каждом отдельном ряде так, что слева должны были стоять самые низкие, а справа самые высокие. Ну а поскольку в армии виноваты всегда слабые (низкие), то наказание было следующим: каждый солдат должен был отжаться столько раз, сколько солдат стоит от него слева выше его ростом.

Оказалось, что все солдаты были разного роста, и многим пришлось отжиматься достаточно много раз. Сержанту стало интересно: сколько же раз в общей сложности пришлось отжаться солдатам?

Помогите ему решить эту задачу!

#### Входные данные

В первой строке входного файла INPUT.TXT записаны два натуральных числа  $N$  и  $K$  ( $2 \leq N \leq 10^4$ ,  $1 \leq K \leq 20$ ) – число солдат в каждом ряде и число рядов. Следующие  $K$  строк файла содержат ровно  $N$  разных натуральных чисел от 1 до  $N$  – рост солдат. Первое число ряда – рост первого солдата (самого левого в ряду), второе – рост второго, и т.д.

#### Выходные данные

В выходной файл OUTPUT.TXT необходимо вывести общее количество отжиманий, которые должны были выполнить солдаты.

#### Примеры

| № | INPUT.TXT                          | OUTPUT.TXT |
|---|------------------------------------|------------|
| 1 | <pre>3 3 1 2 3 2 1 3 3 2 1</pre>   | 4          |
| 2 | <pre>5 2 1 5 2 4 3 2 3 1 5 4</pre> | 7          |

```
#include <bits/stdc++.h>
using namespace std;

using int64 = long long;

// [l, r) segmentida mergesort va inversiyalarni sanash
static int64 merge_count(vector<int>& a, vector<int>& tmp, int l, int r) {
    if (r - l <= 1) return 0;
    int m = (l + r) >> 1;
    int64 ans = merge_count(a, tmp, l, m) + merge_count(a, tmp, m, r);

    int i = l, j = m, k = l;
    while (i < m && j < r) {
        if (a[i] <= a[j]) {
            tmp[k++] = a[i++];
        } else {
            tmp[k++] = a[j++];
        }
    }
}
```



```

        ans += (m - i);           // chapda qolganlarning barchasi a[j]-dan
katta
    }
}
while (i < m) tmp[k++] = a[i++];
while (j < r) tmp[k++] = a[j++];
for (int t = l; t < r; ++t) a[t] = tmp[t];
return ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int N, K;
    if (!(cin >> N >> K)) return 0;

    vector<int> a(N), tmp(N);
    long long total = 0;

    for (int row = 0; row < K; ++row) {
        for (int i = 0; i < N; ++i) cin >> a[i];    // 1..N bo'lgan turli sonlar
        total += merge_count(a, tmp, 0, N);
    }

    cout << total << '\n';
    return 0;
}

```

```

#include <bits/stdc++.h>
using namespace std;

using int64 = long long;

// [l, r) kesmada inversiyalarni sanaydi va a[l..r) ni tartiblaydi
static int64 count_inv(vector<int>& a, int l, int r) {
    if (r - l <= 1) return 0;
    int m = (l + r) >> 1;
    int64 ans = count_inv(a, l, m) + count_inv(a, m, r); // chap va o'ng yarmalar
    tartiblanadi

    // Kesishuvchi juftliklar: chap va o'ng yarmalar HOZIR tartiblangan
    int i = l, j = m;
    while (i < m && j < r) {
        if (a[i] <= a[j]) {
            ++i;
        } else {
            ans += (m - i); // a[i] > a[j] bo'lsa, i..m-1 hammasi > a[j]
        }
    }
}

```

```

        ++j;
    }
}

// Endi ikkisini bitta tartiblangan segmentga birlashtiramiz
inplace_merge(a.begin() + l, a.begin() + m, a.begin() + r);
return ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int N, K;
    if (!(cin >> N >> K)) return 0;

    vector<int> a(N);
    long long total = 0;

    for (int row = 0; row < K; ++row) {
        for (int i = 0; i < N; ++i) cin >> a[i]; // 1..N (takrorlanmaydi)
        total += count_inv(a, 0, N);
    }

    cout << total << '\n';
    return 0;
}

```

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;

ll merge(vector<int>& v, int left, int mid, int right){
    int l = left, r = mid + 1;
    vector<int>temp;
    ll cnt = 0;
    while(l <= mid && r <= right){
        if(v[l] <= v[r]){
            temp.push_back(v[l++]);
        } else {
            temp.push_back(v[r++]);
            cnt += (mid - l + 1);
        }
    }
    while(l <= mid){
        temp.push_back(v[l++]);
    }
    while(r <= right){

```

```

        temp.push_back(v[r++]);
    }
    for(int i = left; i <= right; i++){
        v[i] = temp[i - left];
    }
    return cnt;
}

ll mergeSort(vector<int>& v, int left, int right){
    if(left >= right) return 0;
    int mid = (left + right) / 2;
    ll cnt = mergeSort(v, left, mid);
    cnt += mergeSort(v, mid + 1, right);
    cnt += merge(v, left, mid, right);
    return cnt;
}

int main()
{
    ios_base::sync_with_stdio(0);
    cin.tie(0);
    cout.tie(0);

    int n, k;
    ll cnt = 0;
    cin >> n >> k;
    vector<int>v(n);
    for(int i = 0; i < k; i++){
        for(int j = 0; j < n; j++){
            cin >> v[j];
        }
        cnt += mergeSort(v, 0, n - 1);
    }
    cout << cnt;
}

```

## Fenvik daraxti usulida

```

#include <bits/stdc++.h>
using namespace std;

struct Fenwick {
    int n;
    vector<int> bit;
    Fenwick(int n=0): n(n), bit(n+1, 0) {}
    void reset(int n_) { n = n_; bit.assign(n+1, 0); }
    void add(int idx, int val=1) {
        for (; idx <= n; idx += idx & -idx) bit[idx] += val;
    }
}

```

```

    }
    int sumPrefix(int idx) const {
        int s = 0;
        for (; idx > 0; idx -= idx & -idx) s += bit[idx];
        return s;
    }
    int sumRange(int l, int r) const { // optional
        if (r < l) return 0;
        return sumPrefix(r) - sumPrefix(l-1);
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int N, K;
    if (!(cin >> N >> K)) return 0;

    Fenwick fw(N);
    long long total = 0; // jami "otjimaniya"lar

    for (int row = 0; row < K; ++row) {
        fw.reset(N);
        long long seen = 0;
        for (int i = 0; i < N; ++i) {
            int h; cin >> h; // balandlik (1..N, takrorlanmaydi)
            int not_taller = fw.sumPrefix(h); // chapdagilardan balandligi = h
            boʻlganlar
            long long taller_left = seen - not_taller; // balandroqlar soni
            total += taller_left;
            fw.add(h, 1); // hozirgi askarni "koʻrildi" deb
            belgilaymiz
            ++seen;
        }
    }

    cout << total << "\n";
    return 0;
}

```

# Inversiyalar soni



1 dan N gacha bo'lgan sonlar to'plamining ixtiyoriy permutatsiyasi beriladi. Siz berilgan ketma-ketlikdagi inversiyalar sonini topishingiz kerak.

Inversiyalar soni deb quyidagi shartni qanoatlantiruvchi (i, j) juftliklar soniga aytiladi:

- $i < j$
- $array[i] > array[j]$

## Kiruvchi ma'lumotlar:

INPUT.TXT kirish faylining dastlabki satrida bitta butun son,  $N(1 \leq N \leq 10^5)$  soni kiritiladi. Ikkinchi satrda bo'sh joy bilan ajratilgan holda N ta butun son, 1 dan N gacha bo'lgan sonlarning permutatsiyasi kiritiladi.

## Chiquvchi ma'lumotlar:

OUTPUT.TXT chiqish faylida bitta butun son, masala yechimini chop eting.

## Misollar

| # | input.txt                                 | output.txt |
|---|-------------------------------------------|------------|
| 1 | 10<br>7 6 2 4 1 5 10 3 9 8                | 19         |
| 2 | 15<br>2 7 8 13 11 5 1 9 3 14 4 10 6 12 15 | 38         |
| 3 | 11<br>6 10 2 3 9 1 4 7 11 5 8             | 23         |

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

ll merge(vector<int>&arr, int left, int mid, int right){
    int l = left, r = mid + 1;
    ll cnt = 0;

    vector<int>temp;
    while(l <= mid && r <= right){
        if(arr[l] < arr[r]){
            temp.push_back(arr[l++]);
        } else {
            temp.push_back(arr[r++]);
            cnt += mid - l + 1;
        }
    }
    while(l <= mid){
        temp.push_back(arr[l++]);
    }
    while(r <= right){
        temp.push_back(arr[r++]);
    }
}
```

```

        for(int i = left; i <= right; i++){
            arr[i] = temp[i - left];
        }
        return cnt;
    }

    ll mergeSort(vector<int>&arr, int left, int right){
        if(left >= right) return 0;
        int mid = (left + right) / 2;
        ll cnt = mergeSort(arr, left, mid);
        cnt += mergeSort(arr, mid + 1, right);
        cnt += merge(arr, left, mid, right);
        return cnt;
    }

    int main()
    {
        int n;
        cin >> n;
        vector<int>arr(n);
        for(int i = 0; i < n; i++){
            cin >> arr[i];
        }

        ll cnt = mergeSort(arr, 0, n - 1);
        cout << cnt;
    }

```

```

#include <bits/stdc++.h>
using namespace std;

using int64 = long long;

// a[l..r) kesmada inversiyalarni sanaydi va shu bo'lakni tartiblaydi
int64 merge_count(vector<int>& a, vector<int>& tmp, int l, int r) {
    if (r - l <= 1) return 0;
    int m = (l + r) >> 1;
    int64 ans = merge_count(a, tmp, l, m) + merge_count(a, tmp, m, r);

    int i = l, j = m, k = l;
    while (i < m && j < r) {
        if (a[i] <= a[j]) {
            tmp[k++] = a[i++];
        } else {
            tmp[k++] = a[j++];
            ans += (m - i); // chapda qolganlarning hammasi a[j] dan katta
        }
    }
    while (i < m) tmp[k++] = a[i++];
    while (j < r) tmp[k++] = a[j++];
}

```

```

        for (int t = 1; t < r; ++t) a[t] = tmp[t];

        return ans;
    }

    int main() {
        ios::sync_with_stdio(false);
        cin.tie(nullptr);

        int N;
        if (!(cin >> N)) return 0;
        vector<int> a(N);
        for (int i = 0; i < N; ++i) cin >> a[i];

        vector<int> tmp(N);
        cout << merge_count(a, tmp, 0, N) << '\n';
        return 0;
    }

```

## Fenvik daraxti usulida

```

#include <bits/stdc++.h>
using namespace std;

struct Fenwick {
    int n;
    vector<int> bit;    // 1..n
    Fenwick(int n=0): n(n), bit(n+1, 0) {}

    // i ga +v qo'shish
    void add(int i, int v=1) {
        for (; i <= n; i += i & -i) bit[i] += v;
    }

    // 1..i oraliq yig'indisi
    long long sum(int i) const {
        long long s = 0;
        for (; i > 0; i -= i & -i) s += bit[i];
        return s;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

```

```

int N;
if (!(cin >> N)) return 0;

vector<int> a(N);
for (int i = 0; i < N; ++i) cin >> a[i]; // bu - 1..N permutatsiya

Fenwick fw(N);
long long inv = 0;

// O'ngdan chapga yuramiz: har bir a[i] uchun undan kichiklari soni -
sum(a[i]-1)
for (int i = N - 1; i >= 0; --i) {
    inv += fw.sum(a[i] - 1);
    fw.add(a[i], 1);
}

cout << inv << '\n';
return 0;
}

```



# Massiv



$n$  ta elementdan iborat bo'lgan butun sonli  $a$  massiv berilgan. Ushbu massivda quyidagi shartni qanoatlantiruvchi elementlar juftligi sonini aniqlang:

- $i < j$
- $a[i] > 2 * a[j]$

## Kiruvchi ma'lumotlar:

INPUT.TXT kirish faylining birinchi satrida bitta natural son, massiv elementlar soni  $n$  ( $n \leq 10^5$ ). Ikkinchi satrda  $n$  ta butun son massiv elementlari. massiv elementlari qiymati  $[-10^9; 10^9]$  orasida.

## Chiquvchi ma'lumotlar:

OUTPUT.TXT chiqish faylida masalada berilgan shartni qanoatlantiruvchi juftliklar sonini chop eting.

## Misollar

| # | input.txt                       | output.txt |
|---|---------------------------------|------------|
| 1 | 8<br>32 11 37 82 27 15 53 16    | 8          |
| 2 | 8<br>37 37 91 -76 -13 13 -32 32 | 15         |

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

ll merge(vector<int>&arr, int left, int mid, int right){
    int l = left, r = mid + 1;
    ll cnt = 0;
    for(int i = l; i <= mid; i++){
        while(r <= right && arr[i] > 2LL * arr[r]){
            r++;
        }
        cnt += (r - (mid + 1));
    }
    r = mid + 1;
    vector<int>temp;
    while(l <= mid && r <= right){
        if(arr[l] < arr[r]){
            temp.push_back(arr[l++]);
        } else {
            temp.push_back(arr[r++]);
        }
    }
    while(l <= mid){
        temp.push_back(arr[l++]);
    }
}
```

```

while(r <= right){
    temp.push_back(arr[r++]);
}
for(int i = left; i <= right; i++){
    arr[i] = temp[i - left];
}
return cnt;
}

ll mergeSort(vector<int>&arr, int left, int right){
    if(left >= right) return 0;
    int mid = (left + right) / 2;
    ll cnt = mergeSort(arr, left, mid);
    cnt += mergeSort(arr, mid + 1, right);
    cnt += merge(arr, left, mid, right);
    return cnt;
}

int main()
{
    int n;
    cin >> n;
    vector<int>arr(n);
    for(int i = 0; i < n; i++){
        cin >> arr[i];
    }

    ll cnt = mergeSort(arr, 0, n - 1);
    cout << cnt;
}

```

```

#include <bits/stdc++.h>
using namespace std;

using ll = long long;

long long countPairs(vector<long long>& a, int l, int r) {
    if (r - l <= 1) return 0; // [l, r)
    int m = (l + r) / 2;
    long long ans = countPairs(a, l, m) + countPairs(a, m, r);

    // count cross pairs: i in [l, m), j in [m, r)
    int j = m;
    for (int i = l; i < m; ++i) {
        while (j < r && a[i] > 2LL * a[j]) ++j;
        ans += (j - m);
    }

    // merge step
    inplace_merge(a.begin() + l, a.begin() + m, a.begin() + r);
}

```

```

        return ans;
    }

    int main() {
        ios::sync_with_stdio(false);
        cin.tie(nullptr);
        int n;
        if (!(cin >> n)) return 0;
        vector<long long> a(n);
        for (int i = 0; i < n; ++i) cin >> a[i];
        cout << countPairs(a, 0, n) << "\n";
        return 0;
    }

```

## Fenwick daraxti usulida

```

#include <bits/stdc++.h>
using namespace std;

struct Fenwick {
    int n;
    vector<int> bit;           // 1..n
    Fenwick(int n=0): n(n), bit(n+1, 0) {}
    void add(int i, int v=1) { // pozitsiyaga +v qo'shish
        for (; i <= n; i += i & -i) bit[i] += v;
    }
    long long sum(int i) const { // 1..i yig'indisi
        long long s = 0;
        for (; i > 0; i -= i & -i) s += bit[i];
        return s;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    vector<long long> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];

    // Kompressiya uchun qiymatlar ro'yxati: a[i] va 2*a[i]
    vector<long long> all;
    all.reserve(2*n);
    for (int i = 0; i < n; ++i) {
        all.push_back(a[i]);
        all.push_back(2LL * a[i]);
    }
}

```

```

}
sort(all.begin(), all.end());
all.erase(unique(all.begin(), all.end()), all.end());

Fenwick fw((int)all.size());
long long ans = 0;
long long seen = 0; // hozirgacha qo'shilgan elementlar soni

for (int j = 0; j < n; ++j) {
    // > 2*a[j] bo'lgan ko'rilganlar soni = seen - count(<= 2*a[j])
    int idx2 = upper_bound(all.begin(), all.end(), 2LL * a[j]) - all.begin();
    // 0-based count
    long long notGreater = fw.sum(idx2); // <= 2*a[j]
    ans += (seen - notGreater);

    // Hozirgi a[j]ni BITga qo'shamiz
    int pos = int(lower_bound(all.begin(), all.end(), a[j]) - all.begin()) +
1; // 1-based
    fw.add(pos, 1);
    ++seen;
}

cout << ans << '\n';
return 0;
}

```